Lions vs Gazelle

Fitz Koch

2026-03-31

Overview (Goals)

- Reimplementation of Luke and Spector (1996)
- Add visualization
- Potential Extension

Design

Overview

There are four lions chasing a gazelle across a toroidal serengeti. Every turn the gazelle moves 3 units according to a set pattern, and then the lions move 1 unit according to an evolved vector operation tree. Both can move in any direction. If the lions get within one unit of the gazelle at any step, they catch it, and have a fitness loss of zero. If they don't, then their fitness is one minus their closest position at the end of the simulation.

The point of this is to study the emergence of coordination *without communication*, as the lions can't actually talk to one another.

Operators

Name	Args	Group	Description
last	0	0	One unit in last direction. First move: rand
rand-dir	0	0	random normal vec
gazelle	0	0	smallest vector lion to gaz
+	2	0	add two ve

Name	Args	Group	Description
-	1	0	negate
$\times 2$	1	0	multiply magnitude of vect
/2	1	0	divide magnitude of vec
$\rightarrow$ 90	1	0	rotate vector clockwise 9
rand	1	0	given a vector, return vector in the same direction, whose magnitude varies between 0 and original
inv	1	0	invert magnitude…: start with $\| \max \| = \sqrt{\left(\frac{w}{2}\right)^2 + \left(\frac{h}{2}\right)^2}$ and then return $\frac{v}{\|v\|} = (\| \max \| - \|v\|)$.
if-dot	4	0	Evaluate the first and second arguments. If their dot product is greater than or equal to 0, then evaluate and return the third argument, else evaluate and return the fourth argument.
if>=	4	0	Evaluate the first and second arguments. If the magnitude of the first argument is greater than or equal to the magnitude of the second argument, then evaluate and return the third argument, else evaluate and return the fourth argument.
near-est	0	1	vector from lion nearest gazelle to gazelle
lion	0	1	vector from lion to nearest neighbor
rlion	0	1	vector from lion to first lino encountered in clockwise sweep. sweep begins in the direction lion moved last. all minimum are gathered, then find first.
llion	0	1	vector from lion to first lino encountered in counterclockwise sweep. sweep begins in the direction lion moved last. all minimum are gathered, then find first.
lion-n	0	2	A vector from the lion to lion n

Breeding

- clones (same in every brain)
- free breeding (any member to breed with any member)
- restricted: only lion i with lion i, etc.

World and Animals

- toroidal, $n \times n$, with $n = 15$; must remember wrapping
- discrete updates
- Gazelle:
  - randomly placed
  - 3 units per turn
  - follows:

$$-\sum_{v \in V} \frac{v}{\|v\|}(\|max\| - \|v\|)$$

- Lion
  - randomly placed
  - 1 unit per turn
  - goal: get one lion with 1 unit of the gazelle

Math Summary:

- $\|\text{max}\| = \sqrt{\left(\frac{w}{2}\right)^2 + \left(\frac{h}{2}\right)^2}$
- gazelle movement: $-\sum_{v \in V} \frac{v}{\|v\|}(\|\text{max}\| - \|v\|)$

Genetic Programming

Though the paper doesn't really discuss it, at this point genetic programming refers to a system where internal nodes are functions and operators, and leaf nodes are variables/constants. They use `lil-gp`, a C codebase or library or whatever you call them, but I plan on just reimplementing in python. Basically, we build a tree,

- population 500 (of lions/trees? or of prides?)
  - decided **prides**
- maximum tree size: 70
- maximum depth: 17
- crossover (90%)
- reproduction (10%)
- selection: unclear. Claude thinks probably tournament selection, ? But maybe more research required.

Testing

- 100 runs for each combo of sensing and breeding, 100 for each control
- fitness of groups based on average of 5 simulation trials
- 15 steps per sim
- 0 length vector => random movement of one unit.
- 51 generations
- population of 500
- **Fitness**: 0 if closest lion within 1 unit, else distance minus 1. minimization.

Code Design

We are going to evaluate a pride that consists of four separate lions, which we can represent as a class with four separate tree classes. This requires:

Code Design

We are going to evaluate a pride that consists of four separate lions, which we can represent as a class with four separate tree classes. This requires:

0. Basic loop: positions -> contexts -> tree processing -> back to positions
1. A recursive node class
    - operation definition (including required number of children)
    - children
    - method to operate on children (recursively) and return result
    - take in context (from positions)
2. Random tree generation: use the node class to make trees that meet specifications
3. Parent selection: select 900 parents via tournament selection ($N = 7$), taking the best of each tournament.
4. Crossover
    - Individual selection rules by strategy:
        - Cloning: all individuals identical; doesn't matter.
        - Free Breeding: pick at random.
        - Restricted Breeding: only lion i with lion i.
    - Swap subtrees; reject if result exceeds depth 17 or size 70.
5. Mutation: pick random node, replace with a new random subtree.
    - weirdly, somehow, I just made this up. they say reproduction, and I just missed this.
    - reproduction, we just take the best 10%.
6. A world: 15x15, floating point, toroidal.
7. Algorithm: evolutionary loop for 51 generations.
    - First generation: randomly generate 500 prides.
    - Future generations: select parents, apply crossover (90%) and mutation (10%).
    - Evaluation per pride:
        - Run 5 simulations, each up to 15 steps.
        - Each step: update contexts, move gazelle by fear vector, move lions by tree evaluation, check catch condition.
        - Fitness: average of (nearest - 1) across simulations, 0 if caught.

- Cache best pride and positions per generation.

8. After 51 generations, store:

    - Best fitness by generation.
    - Average fitness by generation.
    - List of best prides per generation.
    - Best positions per generation.

Luke, Sean, and Lee Spector. 1996. "Evolving Teamwork and Coordination with Genetic Programming." In *Proceedings of the 1st Annual Conference on Genetic Programming*, 150–56. Cambridge, MA, USA: MIT Press.